

COMPSCI 101 Assignment – The Decryption Game

Due: 11:59pm, Monday 1st June 2026.

Worth: This assignment is marked out of 60 and is worth 4% of your final mark.

Topics covered:

- Using variables
- Printing output
- Manipulating string objects
- Manipulating list objects
- Getting user input
- Using loops
- Defining functions
- Reading text files
- Using dictionaries

Submit the file containing your assignment using the Assignment Dropbox:

<https://adb.auckland.ac.nz/Home/>

INTRODUCTION

For this assignment you will need to implement a simple game of decryption. The game tasks the player with decrypting the consonants in a word. Vowels are the letters "a", "e", "i", "o" and "u". Consonants are the remaining letters of the alphabet. The consonants are represented by non-negative integer codes. The player will need to figure out which consonant each integer code represents. Let's take a look at an example:

16 ^u 18 18 11 8

Here, the player will need to figure out which consonants the codes 8, 11, 16 and 18 represent. In each round of the game, the player can select an integer code they want to decrypt and then enter the consonant they think the code represents. Let's say the player chooses to decrypt the code 18 and enters the letter "p" as the consonant it represents. If this is correct then the word would be updated as follows:

16 ^u ^p ^p 11 8

When the player chooses an incorrect consonant for the code they have selected, the word is not updated. The game allows the player to make up to 10 mistakes. The game ends once the player decrypts the entire word or makes 10 mistakes.

Your program will need to:

- Display a banner.
- Read a text file containing the words that can be used by the game.
- Randomly choose one of these words for the user to decrypt. This is handled by the `get_word()` function which is provided for you.
- Create a dictionary from this word, where the keys are the consonants in the word, and the values are randomly generated non-negative integer codes that will represent them. This is handled by the `get_encryption_dictionary()` function which is provided for you.
- Run the decryption portion of the game. This will involve several rounds. In each round:
 - The current status of the word is displayed.
 - The player selects the integer code they want to decrypt.
 - The player enters the consonant they think the code represents.
 - The game informs the player whether or not their guess is correct.
- The game ends when the player has successfully decrypted the word or has made 10 mistakes.

You will be provided with a skeleton file for the assignment – `Project_Skeleton.py`. The file contains:

- The statement `import random`. This statement is required to randomly select the word to decrypt and to generate the integer codes representing the consonants in the word. Do not remove this statement!
- An implementation of the `get_word()` function. This function takes a list of strings as a parameter and returns a randomly selected string. This string will be the word the player needs to decrypt. Do not remove or alter this function!
- An implementation of the `get_encryption_dictionary()` function. This function takes a single string parameter – the word the player will need to decrypt. The function will create and return a dictionary object where the keys are the consonants in the word, and the values are randomly generated non-negative integer codes that will represent them. Each consonant is represented by a unique non-negative integer code. This dictionary will be central to the decryption game. Do not remove or alter this function!
- A call to the `main()` function. Do not remove this statement!
- Function headers for functions you will need to complete:
 - `print_banner()`
 - `get_words()`
 - `get_consonant_status_dictionary()`
 - `print_puzzle()`
 - `choose_valid_encryption_code()`
 - `choose_valid_consonant()`
 - `play_round()`
 - `play_game()`
 - `main()` – Note that part of its implementation has been provided for you.

The functions that you need to complete are described next.

print_banner() (2 marks)

The `print_banner()` function takes a single string parameter, `username`, that represents the player's username. It uses the player's username to generate the message "The Decryption Game by {username}". It then prints this message inside a banner. Note that the username should be printed in **lowercase**. Also note that the length of the banner is dependent on the length of the username. Not all usernames have the same length.

For example, if the function is called passing it the username "dazh001", then the function would print the following banner:

```
*****
* The Decryption Game by dazh001 *
*****
```

If the function is called passing it the username "abc001", then the function would print the following banner:

```
*****
* The Decryption Game by abc001 *
*****
```

The function **must** print a blank line after the banner

get_words() (2 marks)

The `get_words()` function takes a single string parameter `filename`, representing the name of the text file containing the words that the decryption game can use. The file will consist of one or more lines of text, where each line consists of a single word. All the words are in lowercase. The function will read the text file and create and return a list of the words contained in the file.

get_consonant_status_dictionary() (4 marks)

The `get_consonant_status_dictionary()` function takes a single string parameter called `word`. This parameter represents the word that the player will need to decode.

The function will create and return a dictionary object where the keys are the consonants found in `word`, and the values are the Boolean `False`. This dictionary is used to keep track of the decryption status of the consonants in the word the player has to decrypt. As none of the consonants are initially decrypted, the value for each consonant key is set to `False`. As the game is played, whenever a consonant is successfully decrypted, its corresponding value will be updated to `True`.

For example, if `word = "supply"` then the function would return the dictionary

```
{ 's':False, 'p':False, 'l':False, 'y':False }
```

`print_puzzle()` (8 marks)

The `print_puzzle()` function prints the current status of the word the player has to decrypt. The function takes 3 parameters:

1. `word` – a string representing the word the player has to decrypt.
2. `consonant_status_dict` – a dictionary object where the keys are the consonants in `word` and the values are Booleans – `True` if a consonant has been decrypted and `False` otherwise.
3. `encryption_dict` – a dictionary object where the keys are the consonants in `word`, and the values are randomly generated integer codes that will represent them. Note that each consonant is represented by a unique integer code.

The `print_puzzle()` function will print two lines of output.

1. The first line of output is used to display the letters in `word`. Vowels and the consonants that have been decrypted are displayed so that they are each preceded by a single space character. Consonants that have not been decrypted are represented using the placeholder `'__'` (two underscores). Each letter/placeholder in `word` is separated by a single space.

For example, if the word to decrypt is `'supply'`, then the first line displayed by the `print_puzzle()` function when the game starts will be:

`__ u __ __ __`

Only the vowel `'u'` is initially displayed, with the consonants represented by the placeholder `'__'`. Note, the single space before the letter `'u'` and the space between each letter/placeholder in the word.

Now, let's say the player has correctly decrypted the consonants `'p'` and `'y'`. The first line displayed by the `print_puzzle()` function would then be:

`__ u p p __ y`

Again, note how the letters `'u'`, `'p'`, and `'y'` are preceded by a single space. Also note that the two consonants that have not been decrypted, `'s'` and `'l'`, are still represented by the placeholder `'__'`.

2. The second line of output is used to display the non-negative integers representing the consonants in `word`. Using `'supply'` as the example again, then the `print_puzzle()` function could print out the following:

`16 u 18 18 11 8`

You can see that 16 represents `'s'`, 18 represents `'p'`, 11 represents `'l'` and 8 represents `'y'`. Note that these integer codes are randomly selected; they will differ each time the game is run. However, every consonant in `word` will always be represented by a unique non-negative integer code. Also note that if an integer code is a single digit (as is the case with the code 8), it will be preceded by a single space character.

When the player correctly decrypts a consonant, the integer code for it is no longer displayed on the second line. If the player has correctly decrypted the consonants 'p' and 'y', then the output displayed by the `print_puzzle()` function would be:

u p p y
16 11

Now, only the integer codes for 's' and 'l' are displayed on the second line.

choose_valid_encryption_code() (6 marks)

The `choose_valid_encryption_code()` function takes 2 parameters:

1. `consonant_status_dict` – a dictionary object where the keys are the consonants in the word to decrypt and the values are Booleans – `True` if a consonant has been decrypted and `False` otherwise.
2. `encryption_dict` – a dictionary object where the keys are the consonants in the word to decrypt and the values are randomly generated non-negative integer codes that will represent them. Note that each consonant is represented by a unique non-negative integer code.

The function prompts the player to enter the integer code they want to decrypt and returns the code once the player has entered a valid one. The prompt used should be: "Choose an encryption code to decrypt: "

A code is invalid when:

- The user enters an input that is not a non-negative integer. If this is the case, the player should be informed with the message: "Please enter a non-negative integer for your choice!"
- It is not one of the codes used to represent a consonant in the word to decrypt. If this is the case, the player should be informed with the message: "{chosen_code} is not a valid code!"
- It has already been decrypted. If this is the case, the player should be informed with the message: "This encryption code has already been decrypted!"

While the player has not entered a valid code, they should be informed of their mistake with the appropriate message as discussed previously. Once the player enters a valid code, the function prints: "You have chosen the code: {chosen_code}". The function then returns this code.

choose_valid_consonant() (6 marks)

The `choose_valid_consonant()` function takes a single dictionary object as a parameter called `consonant_status_dict`, where the keys are the consonants in the word to decrypt and the values are Booleans – `True` if a consonant has been decrypted and `False` otherwise.

The function prompts the player to enter a consonant and returns it once the player enters a valid one. You can assume that the player will always enter a single character when prompted. The prompt used should be: "Enter a consonant for the code: "

The player's input is invalid when:

- The character entered is not alphabetical. The player should be informed with the message: "Please enter a consonant only!"
- The character entered is a vowel. The player should be informed with the message: "Please enter a consonant only!"
- The consonant entered has already been decrypted. The player should be informed with the message: "'{chosen_consonant}' has already been decrypted!"

While the player has not entered a valid input, they should be informed of their mistake with the appropriate message as discussed previously. Once the player enters a valid consonant, the function prints: "You have chosen the consonant: '{chosen_letter}'". The function then returns this consonant. Note that the function should be able to handle both lowercase and uppercase consonants.

play_round() (6 marks)

The `play_round()` function takes 5 parameters:

1. `round_number` – an integer representing the round number.
2. `mistakes_remaining` – an integer representing the number of mistakes the player has left remaining.
3. `word` – a string representing the word the player has to decrypt.
4. `consonant_status_dict` – a dictionary object where the keys are the consonants in the word to decrypt and the values are Booleans – True if a consonant has been decrypted and False otherwise.
5. `encryption_dict` – a dictionary object where the keys are the consonants in the word to decrypt and the values are randomly generated integer codes that will represent them. Note that each consonant is represented by a unique integer code.

This function is used to play a single round of the decryption game. The function should:

- Print out the round number followed by how many mistakes the player has left remaining. The format should be:

```
ROUND X - Y mistake(s) remaining:
```

where X is the round number and Y is the number of mistakes remaining. The function then prints a blank line.

- Print the current status of the word the player has to decrypt. This is done by calling the `print_puzzle()` function. The function then prints a blank line.
- Get the player to choose the integer code they want to decrypt. This is done by calling the `choose_valid_encryption_code()` function.
- Get the player to enter the consonant they think this integer code represents. This is done by calling the `choose_valid_consonant()` function

- If the player correctly identifies the consonant represented by the code they have chosen, the function prints:

```
"Well done! The consonant '{chosen_consonant}' corresponds to the
code {chosen_code}!"
```

The function updates the entry for the consonant in the `consonant_status_dict` to `True` as it has now been decrypted. It then returns `True`.

- Otherwise, the function prints:

```
"Bad luck! The consonant '{chosen_consonant}' does not correspond
to the code {chosen_code}!"
```

The function then returns `False`.

Have a look at the following example:

ROUND 1 - 10 mistake(s) remaining:

```

17   u   13 13 10 2

```

Choose an encryption code to decrypt: **23**

23 is not a valid code!

Choose an encryption code to decrypt: **2**

You have chosen the code: 2

Enter a consonant for the code: **9**

Please enter a consonant only!

Enter a consonant for the code: **Y**

You have chosen the consonant: 'y'

Well done! The consonant 'y' corresponds to the code 2!

You can see that this is the first round of the game. The player has 10 mistakes remaining. The function then prints out the current status of the word the player has to decrypt. Note the blank line between the first line and the word status and the blank line following the current word status.

The player initially enters 23 as the code they want to decrypt. This is an invalid code; they get informed of this and are prompted to enter a code again. This time around they enter a valid code – 2.

They are then prompted to enter the consonant they think this code represents. They first enter '9'. This is not a consonant, and the game informs the player of their mistake. They are prompted to enter a consonant again upon which they then enter 'Y' which is correct. The function informs the player of this, updates the entry for 'y' in the `consonant_status_dict` to `True`, and returns `True`.

play_game() (4 marks)

The `play_game()` function takes 3 parameters:

1. `word` – a string representing the word the player has to decrypt.
2. `consonant_status_dict` – a dictionary object where the keys are the consonants in the word to decrypt and the values are Booleans – `True` if a consonant has been decrypted and `False` otherwise.
3. `encryption_dict` – a dictionary object where the keys are the consonants in the word to decrypt and the values are randomly generated integer codes that will represent them. Note that each consonant is represented by a unique integer code.

The function plays the decryption game. While the player has mistakes remaining and they have not decrypted the word, the function:

- Calls the `play_round()` function to play a round of the game.
- Updates the number of mistakes the player has remaining.
- Updates the round number.
- Prints a blank line.

When implementing this function, you will need to determine whether the player has successfully decrypted the word. Think about how you would use the `consonant_status_dict` to do so.

main() (4 marks)

The `main()` function takes no parameters and is used to run the quiz game program. The first two statements, both of them initialization statements, have been provided for you. Update the first statement so that you initialize the variable `username` to your username.

Complete the rest of the function so that it:

1. Calls the `get_words()` function to obtain a list of words from the specified text file.
2. Calls the `get_word()` function to randomly select a word from the list of words obtained previously.
3. Calls the `get_encryption_dictionary()` function to obtain a dictionary object where the keys are the consonants in the word to decrypt, and the values are randomly generated integer codes that will represent them.
4. Calls the `get_consonant_status_dictionary()` function to obtain a dictionary object where the keys are the consonants in the word to decrypt and the values are Booleans – `True` if a consonant has been decrypted and `False` otherwise.
5. Calls the `print_banner()` function to print the game banner.
6. Calls the `play_game()` function to play the decryption game.
7. Prints the message "Congratulations! You have decrypted '{word}' successfully!" if the user has successfully decrypted the word.
8. Prints the message "Bad luck! The word was '{word}'! Try again!" if the player is unsuccessful.

Two examples of the game being played are shown at the end of this document.

Coding Style (20 marks)

The style mark will be determined according to the following principles:

- A docstring should be included at the top of the source file with your username and ID number.
- Variable names are meaningful and relate to the data stored in that variable (e.g. the name accurately describes the data). Variable names should be whole words where possible.
- Function names are meaningful and describe the purpose of the function.
- Clear layout is used to distinguish between different elements, e.g. spaces between operators and operands, one blank line between functions, etc.
- Constructs minimize the complexity of the program -- expressions should be simple, flow of control is not deeply nested.
- Elements are presented consistently, e.g. the standard Python code conventions are followed.
- Functions must be used to break the program up into smaller tasks. As a guideline, functions should typically be no more than 15-20 lines of code.
- Your code must not contain any "while True" or "break" statements.
- Your code must not use global variables.
- Lines of code should not be longer than 79 characters.

Marking Rubric

Implementation	40 marks
<code>get_words()</code>	2 marks
<code>get_consonant_status_dictionary()</code>	4 marks
<code>print_puzzle()</code>	8 marks
<code>choose_valid_encryption_code()</code>	6 marks
<code>choose_valid_consonant()</code>	6 marks
<code>play_round()</code>	6 marks
<code>play_game()</code>	4 marks
<code>main()</code>	4 marks
Style	20 marks
Docstring	2 marks
Variable Names	2 marks
Function Names	2 marks
Function Length 15 - 20 lines	2 marks
Clear Layout	2 marks
Construct Complexity	2 marks
Code Conventions Followed	2 marks
No "break" or "while True"	2 marks
No global variables	2 marks
Lines of Code < 80 characters	2 marks

EXAMPLE 1 OF THE QUIZ GAME BEING PLAYED

* The Decryption Game by abc001 *

ROUND 1 - 10 mistake(s) remaining:

$\frac{\text{u}}{8} \frac{\text{o}}{20} \frac{\text{e}}{8} \frac{\text{e}}{11}$

Choose an encryption code to decrypt: **a**
Please enter a non-negative integer for your choice!
Choose an encryption code to decrypt: **13**
13 is not a valid code!
Choose an encryption code to decrypt: **8**
You have chosen the code: 8
Enter a consonant for the code: **p**
You have chosen the consonant: 'p'
Well done! The consonant 'p' corresponds to the code 8!

ROUND 2 - 10 mistake(s) remaining:

$\frac{\text{p}}{20} \frac{\text{u}}{20} \frac{\text{p}}{20} \frac{\text{o}}{11} \frac{\text{e}}{11}$

Choose an encryption code to decrypt: **20**
You have chosen the code: 20
Enter a consonant for the code: **5**
Please enter a consonant only!
Enter a consonant for the code: **a**
Please enter a consonant only!
Enter a consonant for the code: **s**
You have chosen the consonant: 's'
Bad luck! The consonant 's' does not correspond to the code 20!

ROUND 3 - 9 mistake(s) remaining:

$\frac{\text{p}}{20} \frac{\text{u}}{20} \frac{\text{p}}{20} \frac{\text{o}}{11} \frac{\text{e}}{11}$

Choose an encryption code to decrypt: **20**
You have chosen the code: 20
Enter a consonant for the code: **r**
You have chosen the consonant: 'r'
Well done! The consonant 'r' corresponds to the code 20!

ROUND 4 - 9 mistake(s) remaining:

$\frac{\text{p}}{20} \frac{\text{u}}{20} \frac{\text{r}}{20} \frac{\text{p}}{20} \frac{\text{o}}{11} \frac{\text{e}}{11}$

Choose an encryption code to decrypt: **11**
You have chosen the code: 11
Enter a consonant for the code: **s**
You have chosen the consonant: 's'
Well done! The consonant 's' corresponds to the code 11!
Congratulations! You have decrypted 'purpose' successfully!

EXAMPLE 2 OF THE QUIZ GAME BEING PLAYED

* The Decryption Game by abc001 *

ROUND 1 - 10 mistake(s) remaining:

$\frac{\quad}{5}$ ^e $\frac{\quad}{12}$ $\frac{\quad}{7}$ $\frac{\quad}{1}$ ^u $\frac{\quad}{18}$

Choose an encryption code to decrypt: **5**

You have chosen the code: 5

Enter a consonant for the code: **B**

You have chosen the consonant: 'b'

Bad luck! The consonant 'b' does not correspond to the code 5!

ROUND 2 - 9 mistake(s) remaining:

$\frac{\quad}{5}$ ^e $\frac{\quad}{12}$ $\frac{\quad}{7}$ $\frac{\quad}{1}$ ^u $\frac{\quad}{18}$

Choose an encryption code to decrypt: **12**

You have chosen the code: 12

Enter a consonant for the code: **C**

You have chosen the consonant: 'c'

Bad luck! The consonant 'c' does not correspond to the code 12!

ROUND 3 - 8 mistake(s) remaining:

$\frac{\quad}{5}$ ^e $\frac{\quad}{12}$ $\frac{\quad}{7}$ $\frac{\quad}{1}$ ^u $\frac{\quad}{18}$

Choose an encryption code to decrypt: **7**

You have chosen the code: 7

Enter a consonant for the code: **D**

You have chosen the consonant: 'd'

Bad luck! The consonant 'd' does not correspond to the code 7!

ROUND 4 - 7 mistake(s) remaining:

$\frac{\quad}{5}$ ^e $\frac{\quad}{12}$ $\frac{\quad}{7}$ $\frac{\quad}{1}$ ^u $\frac{\quad}{18}$

Choose an encryption code to decrypt: **1**

You have chosen the code: 1

Enter a consonant for the code: **f**

You have chosen the consonant: 'f'

Bad luck! The consonant 'f' does not correspond to the code 1!

ROUND 5 - 6 mistake(s) remaining:

$\frac{\quad}{5}$ ^e $\frac{\quad}{12}$ $\frac{\quad}{7}$ $\frac{\quad}{1}$ ^u $\frac{\quad}{18}$

Choose an encryption code to decrypt: **18**

You have chosen the code: 18

Enter a consonant for the code: **g**

You have chosen the consonant: 'g'

Bad luck! The consonant 'g' does not correspond to the code 18!

ROUND 6 - 5 mistake(s) remaining:

$\overline{5}$ ^e $\overline{12}$ $\overline{7}$ $\overline{1}$ ^u $\overline{18}$

Choose an encryption code to decrypt: **12**

You have chosen the code: 12

Enter a consonant for the code: **y**

You have chosen the consonant: 'y'

Bad luck! The consonant 'y' does not correspond to the code 12!

ROUND 7 - 4 mistake(s) remaining:

$\overline{5}$ ^e $\overline{12}$ $\overline{7}$ $\overline{1}$ ^u $\overline{18}$

Choose an encryption code to decrypt: **1**

You have chosen the code: 1

Enter a consonant for the code: **r**

You have chosen the consonant: 'r'

Bad luck! The consonant 'r' does not correspond to the code 1!

ROUND 8 - 3 mistake(s) remaining:

$\overline{5}$ ^e $\overline{12}$ $\overline{7}$ $\overline{1}$ ^u $\overline{18}$

Choose an encryption code to decrypt: **5**

You have chosen the code: 5

Enter a consonant for the code: **p**

You have chosen the consonant: 'p'

Bad luck! The consonant 'p' does not correspond to the code 5!

ROUND 9 - 2 mistake(s) remaining:

$\overline{5}$ ^e $\overline{12}$ $\overline{7}$ $\overline{1}$ ^u $\overline{18}$

Choose an encryption code to decrypt: **7**

You have chosen the code: 7

Enter a consonant for the code: **g**

You have chosen the consonant: 'g'

Bad luck! The consonant 'g' does not correspond to the code 7!

ROUND 10 - 1 mistake(s) remaining:

$\overline{5}$ ^e $\overline{12}$ $\overline{7}$ $\overline{1}$ ^u $\overline{18}$

Choose an encryption code to decrypt: **18**

You have chosen the code: 18

Enter a consonant for the code: **p**

You have chosen the consonant: 'p'

Well done! The consonant 'p' corresponds to the code 18!

ROUND 11 - 1 mistake(s) remaining:

5 e 12 7 1 u p

Choose an encryption code to decrypt: **12**

You have chosen the code: 12

Enter a consonant for the code: **d**

You have chosen the consonant: 'd'

Bad luck! The consonant 'd' does not correspond to the code 12!

Bad luck! The word was 'ketchup'! Try again!